VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

DETAILED REPORT SECTION

LOGISTIC REGRESSION MODEL

MACHINE LEARNING

GROUP 11

Supervisors: Ph. D Truong Vinh Lan

o0o

Students: Tran Dang Khoa - 2252363
Phan Tran Quoc Tuan - 2353273
Tran Gia Lam - 2352670
Nguyen Tan Hung - 2352439
Nguyen Huu Cau - 2352129

HO CHI MINH CITY, APRIL 2026

Contents

1 Logistic Regression Methodology **2**
1.1 Model Overview . . . 2
1.1.1 Introduction to Logistic Regression . . . 2
1.1.2 Why Logistic Regression for This Problem? . . . 2
1.2 Mathematical Formulation . . . 2
1.2.1 Binary Logistic Regression (Foundation) . . . 2
1.2.2 Multinomial Logistic Regression (Our Implementation) . . . 2
1.2.3 Loss Function (Cross-Entropy) . . . 3
1.2.4 Optimization Algorithm . . . 3
1.3 Feature Representation . . . 3
1.3.1 Input Features . . . 3
1.3.2 Weight Interpretation . . . 3
1.4 Model Variants and Configurations . . . 3
1.4.1 Regularization Types . . . 3
1.4.2 Class Weighting . . . 4
1.5 Training Procedure . . . 4
1.5.1 Data Preparation . . . 4
1.5.2 Hyperparameter Optimization . . . 4
1.5.3 Final Model Performance . . . 4
1.6 Model Interpretation . . . 5
1.6.1 Learned Coefficients . . . 5
1.6.2 Feature Importance Ranking . . . 5
1.7 Performance Analysis . . . 5
1.7.1 Confusion Matrix Breakdown . . . 5
1.7.2 Per-Class Metrics . . . 6
1.8 Strengths and Limitations . . . 6
1.9 Comparison with Alternatives . . . 6
1.10 Production Considerations . . . 6
1.11 Conclusion . . . 7

1 Logistic Regression Methodology

1.1 Model Overview

1.1.1 Introduction to Logistic Regression

Logistic Regression is a fundamental supervised learning algorithm used for classification tasks. Despite its name containing ”regression,” it is primarily a **classification algorithm** that predicts the probability of an instance belonging to a particular class.

Key Characteristics:

- **Probabilistic Model:** Outputs probabilities rather than hard classifications.
- **Linear Decision Boundary:** Separates classes using linear combinations of features.
- **Interpretable Coefficients:** Each feature has an associated weight showing its importance.
- **Efficient Training:** Convex optimization problem with guaranteed global optimum.
- **Scalable:** Works well with large datasets and high-dimensional features.

1.1.2 Why Logistic Regression for This Problem?

Advantages for Zillow Classification:

1. **Interpretability:** Coefficients directly show feature importance. Business stakeholders can understand ”why” a prediction was made, making regulatory compliance easier.
2. **Baseline Performance:** Serves as a strong baseline for comparison with complex models. It is fast to train, deploy, and often competitive.
3. **Probabilistic Outputs:** Provides confidence scores, useful for decision-making under uncertainty.
4. **Robustness:** Handles multicollinearity with regularization and is less prone to overfitting than decision trees.

Trade-offs:

- Assumes linear separability (may not capture complex patterns).
- Limited expressiveness compared to neural networks.
- Sensitive to feature scaling (requires preprocessing).

1.2 Mathematical Formulation

1.2.1 Binary Logistic Regression (Foundation)

For a binary classification problem with classes $\{0,1\}$, the hypothesis function is defined as:

$$h(\mathbf{x}) = \sigma(z) = \frac{1}{1+e^{-z}} \quad \text{where } z = \mathbf{w}^T\mathbf{x} + b \tag{1}$$

Where $\mathbf{x}$ is the feature vector, $\mathbf{w}$ is the weight vector, b is the bias term, and $\sigma(z)$ is the sigmoid function mapping z to a probability in $[0,1]$.

1.2.2 Multinomial Logistic Regression (Our Implementation)

For our multiclass classification task ($K = 3$: underestimate, accurate, overestimate), we utilize the Softmax function:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)} \quad \text{where } z_k = \mathbf{w}_k^T\mathbf{x} + b_k \tag{2}$$

The decision rule is simply to predict the class with the highest probability:

$$\text{Predicted class} = \arg\max_k P(y = k \mid \mathbf{x}) \tag{3}$$

1.2.3 Loss Function (Cross-Entropy)

The objective is to minimize the negative log-likelihood (cross-entropy loss) over N training samples:

$$L(\mathbf{w}) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log(p_{ik}) \tag{4}$$

To prevent overfitting, we apply **L2 Regularization (Ridge)**:

$$L_{\text{regularized}}(\mathbf{w}) = L(\mathbf{w}) + \lambda\sum_{k=1}^{K}\|\mathbf{w}_k\|^2 \quad \text{where } \lambda = \frac{1}{2C} \tag{5}$$

1.2.4 Optimization Algorithm

We employ **L-BFGS** (Limited-memory Broyden-Fletcher-Goldfarb-Shanno), a Quasi-Newton method that approximates the Hessian matrix. It offers faster convergence than standard gradient descent and handles L2 regularization natively, making it highly suitable for our medium-sized dataset.

1.3 Feature Representation

1.3.1 Input Features

The model utilizes 14 standardized numeric features: $\mathbf{x} = [x_1, x_2, \ldots, x_{14}]^T$. We apply `StandardScaler` to ensure $\mu(x_j) = 0$ and $\sigma(x_j) = 1$. This is critical because unscaled features with large ranges (e.g., `taxvaluedollarcnt`) would dominate the gradient updates.

1.3.2 Weight Interpretation

For each class k, the model learns a weight vector $\mathbf{w}_k$. A positive coefficient ($w_{k,j} > 0$) means increasing feature j increases the log-odds of class k. Conversely, a negative coefficient decreases the log-odds. The magnitude $|w_{k,j}|$ indicates the feature's importance for that specific class.

1.4 Model Variants and Configurations

1.4.1 Regularization Types

Regularization	F1-macro	Non-zero Coefficients
L2 ($C = 0.01$)	**0.4114**	**14/14 (all features)**
L1 ($C = 1.0$)	0.3741	11/14 (sparse)
ElasticNet	0.3560	12/14 (moderately sparse)

Table 1: *Experimental Results on Regularization Types*

Based on these results, L2 regularization was chosen as it yielded the best performance, indicating that all features hold predictive value.

1.4.2 Class Weighting

Imbalanced classes bias the model toward the majority class ("accurate"). We adjusted the loss function by setting `class_weight='balanced'`:

$$w_k = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot n_{\text{samples_}k}} \tag{6}$$

Without balancing, the model predicted the majority class 85% of the time (F1-macro = 0.31). With balancing, predictions were evenly distributed, improving the F1-macro to 0.41 (+32% improvement).

1.5 Training Procedure

1.5.1 Data Preparation

Data was split using a stratified 80/20 train/test split to maintain class distribution. The `StandardScaler` was fitted exclusively on the training set to prevent data leakage.

1.5.2 Hyperparameter Optimization

We utilized Optuna with the Tree-structured Parzen Estimator (TPE) over 5-fold cross-validation. The search space included C (log-uniform), solver (`lbfgs`, `saga`), penalty, and class weight.

Best Hyperparameters Identified (Trial #21):

```
{
    'C': 0.0103,
    'solver': 'lbfgs',
    'penalty': 'l2',
    'class_weight': 'balanced',
    'scaler': 'standard'
}
```

1.5.3 Final Model Performance

The final model was retrained on the full training set using the optimized parameters. It converged in 187 iterations.

- **Test Accuracy:** 44.70%
- **Test F1-macro:** 0.4114
- **Gain over baseline:** +32.97%

1.6 Model Interpretation

1.6.1 Learned Coefficients

Feature	Class 0 (under)	Class 1 (accurate)	Class 2 (over)
taxvaluedollarcnt	+0.612	-0.089	-0.871
calculatedfinishedsquarefeet	+0.556	-0.045	-0.861
latitude	-0.523	+0.102	+0.599
structuretaxvaluedollarcnt	+0.478	-0.067	-0.622
landtaxvaluedollarcnt	+0.441	-0.051	-0.585
lotsizesquarefeet	+0.378	-0.042	-0.501
yearbuilt	-0.312	+0.056	+0.389
taxamount	+0.301	-0.048	-0.412
longitude	+0.278	-0.039	-0.367
bedroomcnt	-0.201	+0.034	+0.289
bathroomcnt	-0.178	+0.029	+0.245

Table 2: *Weight Matrix (Key Features)*

Domain Validation: For `taxvaluedollarcnt`, a high-value property ($+2.5\sigma$) strongly pushes the prediction towards "underestimate" (Class 0). This aligns with real estate domain knowledge: Zillow systematically struggles with luxury properties, often undervaluing them.

1.6.2 Feature Importance Ranking

Rank	Feature	Mean —Coef—	Interpretation
1	taxvaluedollarcnt	0.524	Property value indicator
2	calculatedfinishedsquarefeet	0.487	Size matters
3	latitude	0.412	Location, location, location
4	structuretaxvaluedollarcnt	0.389	Building value vs land
5	landtaxvaluedollarcnt	0.367	Land value signal

Table 3: *Feature Importance Ranking (Top 5)*

1.7 Performance Analysis

1.7.1 Confusion Matrix Breakdown

Actual	Predicted			Total
	under	accurate	over	
under	174	263	55	492
accurate	116	607	278	1001
over	56	302	149	507
Total	346	1172	482	2000

Table 4: *Confusion Matrix on Test Set*

The matrix reveals that the model rarely confuses opposite extremes (under $\leftrightarrow$ over error is only 5.6%). When uncertain at decision boundaries, it conservatively defaults to the majority "accurate" class.

1.7.2 Per-Class Metrics

- **Class 0 (Underestimate):** Precision: 0.50, Recall: 0.35, F1: 0.41.
- **Class 1 (Accurate):** Precision: 0.52, Recall: 0.61, F1: 0.56.
- **Class 2 (Overestimate):** Precision: 0.31, Recall: 0.29, F1: 0.30 (Hardest class to predict).

1.8 Strengths and Limitations

Strengths:

- **Interpretability:** Coefficients offer clear explanations for each prediction.
- **Computational Efficiency:** Training takes < 1 minute; inference takes microseconds.
- **Robustness:** Convex optimization guarantees a global optimum without local minima.

Limitations:

- **Linear Boundaries:** Fails to capture non-linear feature interactions without manual feature engineering (e.g., polynomial features).
- **Assumes Independence:** Multicollinearity can inflate coefficient variance, though regularization mitigates this.

1.9 Comparison with Alternatives

Metric/Aspect	Logistic Regression	Random Forest (Expected)
F1-macro	0.41	0.48 - 0.52
Training Time	< 1 min	5 - 10 min
Interpretability	High (Parametric)	Medium (Feature Importance)
Feature Interactions	No	Yes
Overfitting Risk	Low (with L2 reg)	Medium

Table 5: *Logistic Regression vs. Alternative Ensemble Method*

1.10 Production Considerations

For production, the model and scaler are serialized via `joblib`. A FAST API endpoint can serve real-time predictions efficiently:

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()
model = joblib.load('logistic_regression_model.pkl')
scaler = joblib.load('feature_scaler.pkl')

class PropertyFeatures(BaseModel):
    bathroomcnt: float
    bedroomcnt: float
    taxvaluedollarcnt: float
    # ... other features

@app.post("/predict")
def predict(property: PropertyFeatures):
    features = list(property.dict().values())
    features_scaled = scaler.transform([features])
    pred = model.predict(features_scaled)[0]
    probs = model.predict_proba(features_scaled)[0]
```

```
    return {
        "class": int(pred),
        "confidence": float(probs[pred])
    }
```

1.11 Conclusion

Logistic Regression proved to be an effective baseline model for the Zillow classification task, achieving a 41% F1-macro score. Its primary value lies in its interpretability and computational efficiency. The learned coefficients align with domain knowledge, showing that property value and size dominate the prediction of log-errors. While non-linear models like Random Forest may ultimately achieve higher accuracy, Logistic Regression provides a transparent, robust benchmark suitable for production environments where explainability is paramount.